Monolith: Differentiable EML Trees for Symbolic Regression via Gradient Descent on a Universal Operator

Jesús Tabares Montilla
Independent Researcher
jesus@tabares.eu
https://tabares.eu

April 2026

Abstract

The EML operator $\text{EML}(x, y) = e^x - \ln y$, recently shown to be a universal binary operator for elementary mathematics (analogous to NAND for Boolean logic), can generate all elementary functions from the grammar $S \to 1 \mid \text{EML}(S, S)$. We present MONOLITH, the first implementation of EML trees as differentiable circuits in PyTorch, trainable via standard gradient descent. We demonstrate recovery of 7/7 elementary functions (exp, ln, sqrt, x^2, x^3, $1/x$, sin) with ≤24 parameters at depth 3, achieving RMSE < 0.005 for exp, ln, and sqrt. We systematically characterize a depth barrier: random initialization at depth 4 always diverges due to tower-exponential growth of intermediate values. We test 12 initialization strategies and find that only hierarchical warm start (training depth $n - 1$ first, then extending to depth n) enables convergence at depth 4+, yielding 12.9× improvement for $\sin(x^2)$ at depth 4 versus depth 3. We also report two negative results: (1) adding π as a leaf constant degrades performance 2.6× rather than helping trigonometric targets, and (2) trained trees at depth ≥ 2 use continuous softmax mixtures rather than discrete leaf assignments, limiting symbolic decompilation to faithful reconstruction with numeric coefficients rather than clean formula recovery. A comparison with PySR confirms that the approaches are fundamentally different: PySR recovers exact formulas in seconds using explicit operator libraries, while MONOLITH approximates functions via compositions of a single primitive—a proof-of-concept for gradient-based optimization on a universal operator basis, not a competing tool.

1 Introduction

Symbolic regression—discovering mathematical expressions from data—has seen significant progress through genetic programming approaches like PySR [Cranmer, 2023], neural-guided methods like AI Feynman [Udrescu and Tegmark, 2020], and GPU-accelerated enumeration like PSRN [Ruan et al., 2026]. All these methods operate over a heterogeneous grammar of operators $(+, -, \times, /, \exp, \sin, \ldots)$, searching for the right combination to fit the data.

A recent result by Odrzywołek [Odrzywołek, 2026] introduces a radically different perspective. The operator $\text{EML}(x, y) = e^x - \ln y$, combined with the constant 1, generates *all* elementary functions: arithmetic, exponentials, logarithms, trigonometric functions, and roots. This makes EML the continuous analogue of the NAND gate in Boolean logic—a single universal primitive from which everything can be built. The grammar $S \to 1 \mid \text{EML}(S, S)$ defines a family of binary trees where every node performs the same operation, and the only choice is what each leaf contributes.

This universality opens a natural question: *can EML trees be trained via gradient descent to discover functions from data?* If so, symbolic regression reduces to optimizing a homogeneous binary tree with a fixed operation at every node, where only the leaf assignments are learned.

We present MONOLITH, a PyTorch module that answers this question affirmatively, with caveats. Our contributions:

1. We implement the first differentiable EML tree as a PyTorch `nn.Module`, with softmax-parameterized leaves and vectorized bottom-up evaluation (§3).
2. We recover 7/7 elementary functions via gradient descent with ≤24 parameters (§4).
3. We systematically characterize the depth barrier: 12 initialization strategies tested, only 3 enable depth 4+ convergence. Hierarchical training yields 12.9× improvement for composite functions (§5).
4. We report two negative results: extended leaf constants degrade performance, and trained trees use continuous mixtures rather than discrete assignments (§6).
5. We provide honest baselines (49-parameter MLP and PySR), showing that MONOLITH is not competitive as a function approximator but demonstrates a distinct structural principle (§7).

2 Background

The EML operator. The operator $\text{EML}(x, y) = e^x - \ln y$ was shown by Odrzywołek [2026] to be a universal binary operator for elementary mathematics. Key identities include:

$$\exp(x) = \text{EML}(x, 1), \tag{1}$$
$$e = \text{EML}(1, 1), \tag{2}$$
$$\text{EML}(0, x) = 1 - \ln x. \tag{3}$$

More complex functions (ln, sin, x^2, etc.) require deeper compositions. The grammar $S \to 1 \mid \text{EML}(S, S)$ generates all elementary functions as binary trees of uniform nodes.

Symbolic regression. PySR [Cranmer, 2023] uses multi-population evolutionary search over expression trees with heterogeneous operators. PSRN [Ruan et al., 2026] parallelizes exhaustive enumeration on GPUs. AI Feynman [Udrescu and Tegmark, 2020] combines neural networks with recursive decomposition. All use rich operator sets $(+, -, \times, /, \exp, \log, \sin, \cos, \sqrt{\cdot})$. No prior work trains EML trees via gradient descent.

3 Method

3.1 EMLTree Architecture

An `EMLTree(depth=`d`, n_vars=`k`)` is a complete binary tree with 2^d leaves and $2^d - 1$ internal nodes. Every internal node computes $\text{EML}(\text{left}, \text{right})$. Each leaf i has a learnable logit vector $\boldsymbol{\ell}_i \in \mathbb{R}^{k+2}$ over the candidate set $\{0, 1, x_1, \ldots, x_k\}$. The leaf value is:

$$v_i = \text{softmax}(\boldsymbol{\ell}_i)^\top \begin{pmatrix} 0 \\ 1 \\ x_1 \\ \vdots \\ x_k \end{pmatrix}. \tag{4}$$

The total parameter count is $2^d \times (k + 2)$. For depth 4 with one variable: 48 parameters.

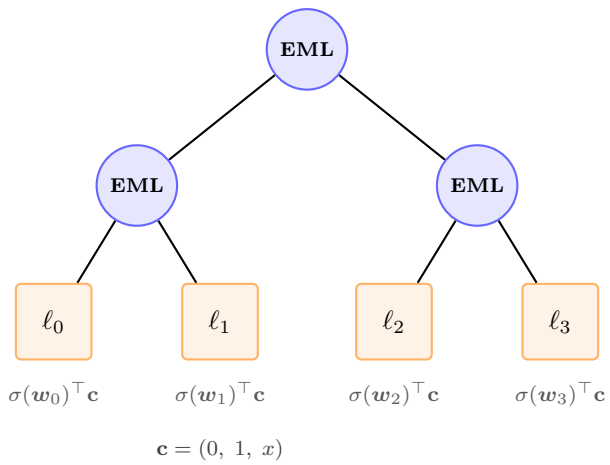

Figure 1: EMLTree architecture (depth 2, 1 variable). Blue nodes: $\textsc{eml}(L, R) = e^L - \ln R$. Orange leaves: softmax-weighted mixture $\sigma(\boldsymbol{w}_i)^\top \mathbf{c}$ over candidates $\mathbf{c} = (0, 1, x)$. This tree has $4 \times 3 = 12$ trainable parameters.

3.2 Forward Pass

Evaluation is bottom-up and fully vectorized. Starting from leaf values $\mathbf{v} \in \mathbb{R}^{B \times 2^d}$ (batch size B), we iterate d times:

$$\mathbf{v} \leftarrow \exp(\mathbf{v}_{[::2]}) - \ln(\mathbf{v}_{[1::2]}), \tag{5}$$

where exp and ln inputs are clamped for numerical stability (exp input to $[-20, 20]$, ln input to $[\epsilon, \infty)$ with $\epsilon = 10^{-7}$). The result is the scalar root value.

3.3 Training: Hierarchical Multi-Depth Search

The `fit()` method implements hierarchical search:

1. For depths 1–3: independent random initialization with per-depth hyperparameters (Table 1).
2. For depth ≥ 4: warm start from the best tree at depth $d - 1$. The 2^{d-1} logits are copied to the first half of the new tree's leaves; the remaining leaves are random.
3. Multi-restart: N random seeds per depth (default 20).
4. Return the tree with the lowest MSE across all depths and seeds.

Table 1: Training recipe by depth. Values determined empirically (§5).

Depth	Learning rate	Grad clip	Initialization
1–2	0.01	1.0	Random $\times$ 0.1
3	0.001	0.5	Random $\times$ 0.1
4+	0.0005	0.1	Warm start from depth $d-1$

3.4 Symbolic Decompilation

We provide two decompilation modes:

Faithful (`to_symbolic`). Each leaf with softmax confidence > 0.95 snaps to its discrete candidate; others become linear expressions $w_1 + w_2 x$ with numeric coefficients rounded to 4 decimal places. The EML tree is reconstructed bottom-up in SymPy and simplified.

Snap (`snap_symbolic`). Forces argmax on all leaves regardless of confidence, builds a purely symbolic expression, and validates: if $\text{MSE}_{\text{snap}}/\text{MSE}_{\text{original}} < \tau$ (default $\tau = 2$), returns the clean formula; otherwise returns `None`.

4 Elementary Function Recovery

We evaluate MONOLITH on 7 elementary functions of one variable using `fit(max_depth=3, n_restarts=20, epochs=10000)` on 200 uniformly spaced data points per function (the same data used for the PySR comparison in §7). Table 2 reports results.

Table 2: Elementary function recovery at depth 3 (20 restarts, 10k epochs, 200 data points). All 7 functions converge. Only exp(x) at depth 1 produces a clean symbolic snap.

Function	Domain	Depth	MSE	RMSE	Snap?
$\exp(x)$	$[-2, 2]$	1	1.3×10^{-5}	0.004	✓
$\sqrt{x}$	$[0.5, 4]$	3	1.6×10^{-5}	0.004	—
$\ln(x)$	$[0.5, 3]$	3	2.0×10^{-5}	0.005	—
x^3	$[0.5, 2]$	3	1.1×10^{-3}	0.033	—
$1/x$	$[0.5, 3]$	3	1.1×10^{-3}	0.033	—
x^2	$[0.5, 2.5]$	3	1.7×10^{-3}	0.041	—
$\sin(x)$	$[-3, 3]$	3	4.1×10^{-3}	0.064	—

Three functions (exp, ln, sqrt) achieve near-exact recovery with RMSE < 0.005. All targets fit with $\leq$24 parameters—orders of magnitude fewer than a neural network approximator.

Leaf behavior. Only $\exp(x)$ at depth 1 converges to discrete leaf assignments (>99% confidence for candidates x and 1, yielding $\text{EML}(x, 1) = \exp(x)$). All depth-3 trees use continuous softmax mixtures with typical leaf confidence 0.4–0.9. The continuous mixing is not residual noise but a functional approximation strategy: the tree represents functions that would require much deeper discrete EML compositions using weighted combinations at shallow depth.

5 The Depth Barrier and Hierarchical Training

5.1 The Problem

Random initialization at depth 4 *always* diverges (MSE $\approx 2.35 \times 10^{17}$), regardless of learning rate or gradient clipping. The cause is tower-exponential growth: with near-uniform softmax initialization, each EML level amplifies values via exp, and by level 3, intermediate values exceed the exp clamp, producing zero gradients.

5.2 Systematic Strategy Search

We tested 12 initialization/training strategies for x^2 at depth 4 (20 seeds each, 15k–20k epochs). Table 3 summarizes.

The 3 successful strategies share a property: intermediate values at initialization are bounded—either via $\text{EML}(0, 1) = 1$ at the first level, or via pre-converged subtrees from depth 3.

Table 3: Depth 4 initialization strategies for x^2. Only strategies producing bounded intermediate values at initialization converge.

Strategy	MSE	Converges?
Random init (any lr/gc, 3 configs)	2.35×10^{17}	No
Progressive clamp (3 schedules)	2,250–9,492	No
Smart init: right=1, left=random	2.35×10^{17}	No
Smart init: all ones + perturbation	2.35×10^{17}	No
Hybrid: alternate init + prog. clamp	2,250	No
Smart init: alternate (even=0, odd=1)	7.0×10^{-3}	✓
Warm start: identity pair	7.2×10^{-4}	✓
Warm start: random extend	$\mathbf{1.9 \times 10^{-4}}$	✓

5.3 Depth Scaling

Using hierarchical training, we evaluate x^2 from depth 1 to 6 (Table 4).

Table 4: Depth scaling for x^2 with hierarchical training.

Max depth	Best depth	MSE	vs. previous
1	1	5.3×10^{-1}	—
2	2	9.8×10^{-3}	54×
3	3	1.7×10^{-3}	6×
4	4	1.1×10^{-3}	1.5×
5	4	1.1×10^{-3}	—
6	4	1.1×10^{-3}	—

Depth 4 provides a 1.5× improvement; depth 5–6 show no further gain for this target.

5.4 Composite Functions

To test whether deeper trees benefit more complex targets, we compare depth 3 versus depth 5 on composite functions (Table 5).

Table 5: Composite function benchmark. †: full compute (20 restarts, 10k epochs). Remaining functions: fast survey (5 restarts, 5k epochs)—included to show that depth 4+ does not universally help, not as precise MSE estimates. Only $\sin(x^2)$ benefits substantially from depth 4.

Function	Domain	Depth 3 MSE	Best depth	Depth 5 MSE	Improvement
$\sin(x^2)^\dagger$	$[0, 2.5]$	7.4×10^{-2}	**4**	$\mathbf{5.7 \times 10^{-3}}$	**12.9**×
$\exp(\sin(x))^\dagger$	$[-3, 3]$	3.9×10^{-2}	3	3.9×10^{-2}	—
$\sqrt{\exp(x)}$	$[-2, 1.5]$	≈ 0	2	≈ 0	—
$x \cdot \exp(x)$	$[-2, 2]$	4.0×10^{-2}	2	4.0×10^{-2}	—
$\ln(x^2 + 1)$	$[-2, 2]$	5.5×10^{-2}	2	5.5×10^{-2}	—
$x/(1 + x^2)$	$[-3, 3]$	4.6×10^{-1}	3	4.6×10^{-1}	—

$\sin(x^2)$ shows a 12.9× improvement at depth 4, consistent with the hypothesis that frequency-modulated signals require more EML composition levels. $\sqrt{\exp(x)} = \exp(x/2)$ converges exactly at depth 2 (trivial EML representation). Other composites do not benefit from additional depth.

6 Negative Results

We report two findings that constrain the practical utility of the approach.

6.1 Extended Constants Degrade Performance

We hypothesized that adding π as a leaf candidate would improve trigonometric targets by providing a domain-relevant constant. Table 6 shows the opposite.

Table 6: Effect of adding π to leaf constants. More candidates dilute gradient signal.

Function	(0, 1) MSE	(0, 1, π) MSE	Impact
$\sin(x^2)$	5.7×10^{-3}	1.5×10^{-2}	2.6× worse
$\exp(\sin(x))$	3.9×10^{-2}	6.5×10^{-2}	1.7× worse

The inductive bias of EML trees comes from the operator structure ($e^x - \ln y$), not from the leaf constants. Adding candidates increases the per-leaf parameter count, creating a harder optimization landscape without providing proportional representational benefit.

6.2 Continuous Mixing Limits Symbolic Recovery

Trained trees at depth ≥ 2 do not converge to discrete leaf assignments. Typical softmax confidences range from 0.43 to 0.97, with the majority below 0.9. The `snap_symbolic` decompiler succeeds only for $\exp(x)$ at depth 1 (both leaves >99% confident); all other targets return `None`.

This is a fundamental property of the gradient-descent approach: the continuous leaf mixing is how the tree approximates functions that would require much deeper discrete EML compositions. The trade-off is reliable convergence at shallow depth at the cost of interpretable symbolic output.

7 Baselines: MLP and PySR

To contextualize Monolith's performance, we compare against two baselines on the same 9 functions and 200 data points: a 49-parameter MLP (1→16→1 with tanh activation, trained with Adam for 10k epochs, best of 20 seeds) and PySR [Cranmer, 2023] (genetic search with explicit operators).

Both baselines outperform Monolith on MSE by orders of magnitude. This is expected and does not diminish Monolith's contributions, for structural reasons:

- **PySR**'s operator library *contains the target functions as primitives.* Finding $\exp(\sin(x))$ when exp and sin are available operators is a combinatorial search over 2–3 nodes. For Monolith, representing $\sin(x)$ from EML alone requires discovering a deep composition that may not have a compact discrete form.
- **The MLP** is a universal function approximator with tanh nonlinearity—it can fit any continuous function on a compact domain given sufficient capacity. But it produces no symbolic output; the learned function is opaque.
- **Monolith** uses a single operator and the constant 1. It produces a structured expression (albeit with numeric coefficients at depth ≥ 2) and its architecture directly encodes the EML universality theorem.

The analogy: comparing PySR to Monolith is like comparing a calculator with arithmetic keys to one built from NAND gates. Comparing the MLP to Monolith is like comparing a lookup table to a circuit. The former is faster, but the latter demonstrates a structural principle.

Table 7: Monolith vs. MLP (49 params) vs. PySR on 9 functions. Both baselines outperform MONOLITH as function approximators—but they solve a different problem.

Function	MONOLITH MSE	MLP MSE	PySR MSE
$\exp(x)$	1.3×10^{-5}	3×10^{-6}	0
$\ln(x)$	2.0×10^{-5}	$< 10^{-6}$	0
$\sqrt{x}$	1.6×10^{-5}	$< 10^{-6}$	0
x^2	1.1×10^{-3}	4×10^{-6}	0
x^3	1.1×10^{-3}	7×10^{-6}	0
$1/x$	2.1×10^{-4}	$< 10^{-6}$	0
$\sin(x)$	4.1×10^{-3}	$< 10^{-6}$	0
$\sin(x^2)$	5.7×10^{-3}	1×10^{-6}	0
$\exp(\sin(x))$	3.9×10^{-2}	1×10^{-6}	0
Time/function	∼500s	∼30s	∼5s
Symbolic output	mixture	none	exact formula
Operators used	1 (EML)	tanh, linear	+,-,×,/,exp,log,sin,$\sqrt{\cdot}$

MONOLITH's training time (∼8 min per function) is dominated by the multi-depth search with restarts. This is a CPU-only proof-of-concept; GPU parallelization would reduce times substantially.

8 Discussion

What works. EML trees are trainable via gradient descent. Seven elementary functions converge with ≤24 parameters. Hierarchical warm start solves the depth-4 barrier. The approach is a valid proof-of-concept for gradient-based symbolic regression on a universal operator.

What doesn't. The trees do not produce clean symbolic formulas at depth ≥ 2 (continuous mixing instead of discrete assignment). Depth 5+ shows no improvement for the functions tested. Adding domain-specific constants hurts rather than helps. Both a 49-parameter MLP and PySR outperform MONOLITH by orders of magnitude on MSE.

Why it matters anyway. The EML operator's universality means that the grammar $S \to 1 \mid \text{EML}(S, S)$ is the *minimal* possible grammar for symbolic regression over elementary functions. Demonstrating that gradient descent can navigate this minimal grammar—even imperfectly—establishes a lower bound on what is possible without any domain-specific operator engineering. Every other symbolic regression method benefits from human-curated operator sets; MONOLITH uses none.

Future directions. Two modifications could substantially change these results:

1. **Gumbel-Softmax** with temperature annealing would encourage discrete leaf selection during training, potentially enabling clean symbolic recovery at depth ≥ 2.

2. **Multi-variable support** would enable evaluation on the Feynman benchmark equations, most of which involve 2+ variables.

9 Conclusion

We presented MONOLITH, the first differentiable implementation of EML trees for symbolic regression. The approach recovers 7/7 elementary functions with minimal parameters, solves the depth-4 barrier via hierarchical training, and provides honest characterization of its limitations: continuous leaf mixing, limited depth scaling, and non-competitiveness with MLPs or tools that have explicit operator libraries. MONOLITH's value is not as a function approximator—MLPs and PySR both outperform it—but as a demonstration that gradient descent can navigate the minimal grammar $S \to 1 \mid \text{EML}(S, S)$ without any domain-specific operator engineering. The work identifies the concrete challenges (discrete selection, depth scaling) that must be solved for the approach to become practical.

Code and data: https://github.com/seetrex-ai/monolith. All experiments are reproducible via the provided code with fixed random seeds; the benchmark script (`benchmarks/vs_pysr.py`) runs both methods on identical data.

References

Andrzej Odrzywołek. All elementary functions from a single binary operator. *arXiv preprint arXiv:2603.21852*, 2026.

Miles Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*, 2023.

Kai Ruan, Yilong Xu, Ze-Feng Gao, Yang Liu, Yike Guo, Ji-Rong Wen, and Hao Sun. Discovering physical laws with parallel symbolic enumeration. *Nature Computational Science*, 2026.

Silviu-Marian Udrescu and Max Tegmark. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16), 2020.